

Fully Differentiable GNN-based PDE Solver

Applications to Poisson + NS equations

1

Why Gen-FVGN anyways?

Introduction

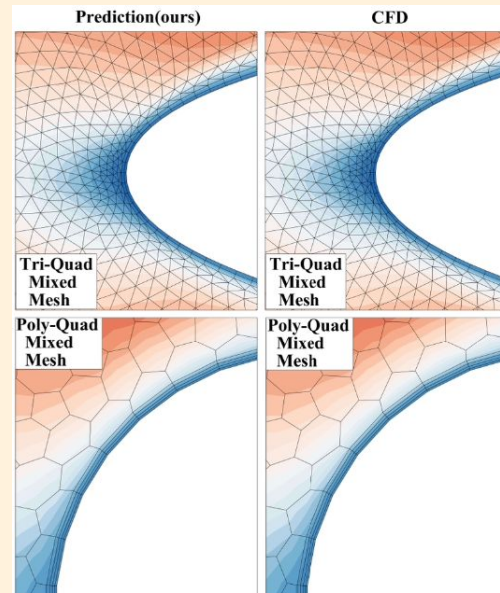
This paper introduces a computational framework using the Finite Volume Method (FVM) and Graph Neural Networks (GNNs) to address challenges that arise in traditional Physics-Informed Neural Networks (PINNs). Gen-FVGN is a **fully differentiable**, **unsupervised** PDE solver i.e. Poisson and Navier-Stokes equations, designed to work directly on unstructured meshes.

OBJECTIVES:

1. Eliminate reliance on **pre-computed training data**
2. Handle **multiple PDEs**, boundary conditions, and source terms in **one model**
3. Achieve high accuracy on **complex geometries** and **irregular grids**

INNOVATIONS:

1. Differentiable FVM discretization with **WLSQ gradient reconstruction**
2. **Vertex-centered** variable layout for natural boundary enforcement
3. GPU-optimized architecture with flexible, **graph-based data structures**



NN Architecture

Gen-FVGN Structure: **Encoder** → **Processor** → **Decoder**:

1. Designed for **unsupervised PDE solving on unstructured meshes**
2. Fully **differentiable**, enables direct FVM loss computation

1 ENCODER: Encodes mesh and physical field into a higher dimensional feature space:

- Vertex Encoder with ϕ_v inputs $[u, v, p]$: the one-hot node type, θ_{pde} that outputs a latent feature vector $v' \in \mathbb{R}^{128}$
- Edge Encoder ϕ_e with difference of adjacent cell units in relative coordinates. And outputs a latent feature vector $e' \in \mathbb{R}^{128}$

2 PROCESSOR: (Message Passing Layer)

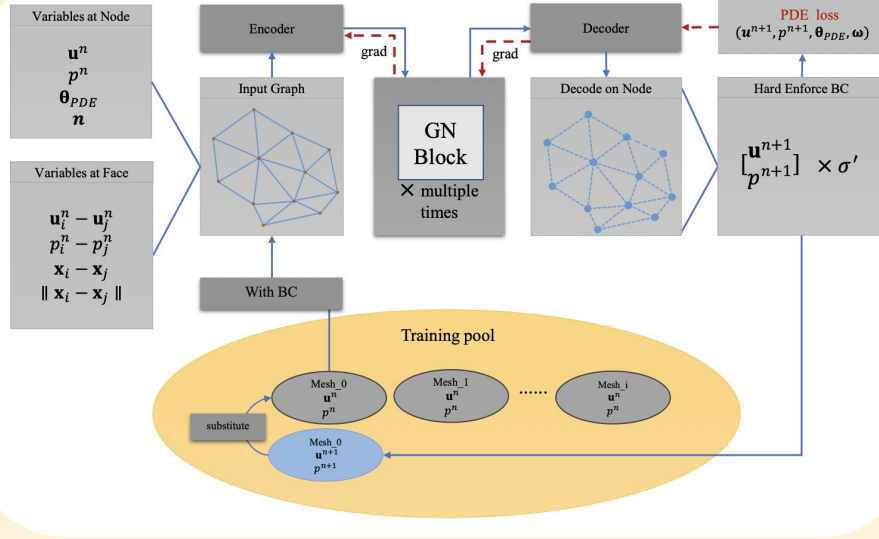
Multiple GN Blocks with two-step aggregations

1. Edge → Cell Center
2. Cell Center → vertex

- Then just update node and edge feature using MLPs with 2 hidden layers (128 units), SiLU activations, and LayerNorm.
- Variables are vertex-centered, but PDE losses are cell-centered

3 DECODER: Maps the latent vertex features → predicted physical fields. Output: $[u, v, p]$

- And uses an equation type mask to zero-out unused channels.
- Uses 2-layer MLP (SiLU, no LayerNorm)



NN Architecture Cont.

4 MORE INPUT DETAILS + PREPROCESSING

NN Architecture Cont.

5 TRAINING DETAILS

Uses **one-step iterative training**. Which entails

- predicting the flow field of the next time step with flow field from the previous step. i.e. $[\mathbf{u}, \mathbf{p}^n] \rightarrow [\mathbf{u}^{n+1}, \mathbf{p}^{n+1}]$
- We use an explicit-implicit format scheme for stability and accuracy (see pg. 12)

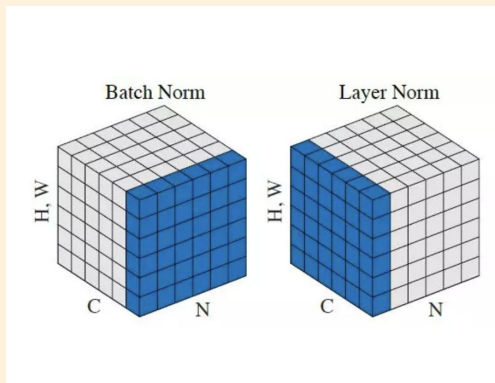
Gen-FVGN maintains a training pool $\mathcal{G} = \{G_1^n, G_2^n, \dots\}$ of meshes which contain the metadata on the mesh G (vertices, edges, cells), the current solutions $\mathbf{u}/\mathbf{v}^n, \mathbf{p}^n$, BCs, and the PDE tag θ_{PDE} .

- At each iteration, a random mini-batch is sampled and passed through on the forward pass to the encoder, GN block, and decoder.
- PDE loss is computed from the decoded $[\mathbf{u}^{n+1}, \mathbf{p}^{n+1}]$ and then backpropagate to update the weights. This is then returned to the pool for the next round.
- One randomly chosen mesh is also reset to encourage diverse learning of BCs and source terms.

6 LOSS + OPTIMIZATION

-

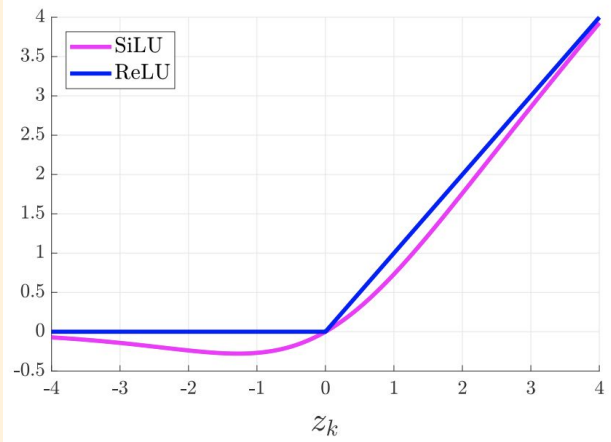
LayerNorm and SiLU



The SiLU activation is used and is the swish function. It is defined as follows:

$$\text{SiLU}(x) = x \cdot \sigma(x)$$

where $\sigma(x)$ is the logistic sigmoid function. As for LayerNorm, we use them to reduce the likelihood of encountering the vanishing gradient or exploding gradient problems, but don't quite eliminate them.



LayerNorm computes the mean and variance over all hidden units. It is much easier to parallelize than **BatchNorm** which is useful in RNNs.

FLOWCHART



Code Environment

Code Environment

As pointed out, since the Gen-FVGN repo employs `torch.compile` for *compiler backend optimization* and *runtime library dependencies*. Thus, we need to download, if on Windows, MSVC (Microsoft Visual C++). We can do this as follows:

1. Install [Visual Studio Code](#).
2. Install the [C/C++ extension for VS Code](#). You can install this extension by searching for 'c++' in the extensions view on the left hand bar of the IDE.
3. Install the MSVC compiler toolset. If you have a recent version of Visual Studio, open the Visual Studio Installer from the Windows Start menu and verify that the C++ workload is checked. If it's not installed, then check the box and select the Modify button in the installer.

Next, you will have to ensure that you download miniconda3 to ensure that conda is available as a python environment tool. You can download it [here](#). Next, in the powershell command prompt in VSCode, ensure you are using the base miniconda3. For **torchvision**, it is ESSENTIAL that you are running python 3.11.0! Now install the rest of the dependencies in the terminal.

```
conda create -n FVGN-pt2.5 python==3.11 # Create a new conda environment and specify the Python version
conda activate FVGN-pt2.5 # Activate the Python environment
conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c nvidia # Install pytorch and torchvision
pip install scipy -i https://pypi.tuna.tsinghua.edu.cn/simple # Pre-install scipy to avoid potential issues
pip install sympy==1.13.1 -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install torch_geometric -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install --no-index pyg_lib torch_scatter torch_sparse torch_cluster torch_spline_conv -f https://github.com/pyg-team/pytorch_geometric/releases/download/2.5.0/
pip install -r src/requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

Source Code Files

When you are about to pre-train Adam using the line: `python stc/pre_train_Adam.py`, you need to change the file called `utils` in the `src` folder to `Utils`.

- Errors might show up

The other issue I had was the use of lambda variables in `Graph_loader.py`. They seemed to be conflicting with dependencies and multiprocessing. So outside of the `Data_Pool` class, you will need to explicitly write an identity method.

```
def identity_collate(x):  
    return x
```

After, that it should just run, but it will take time. Just follow the `README.md` on GitHub.